T1A 鼠标（GT_LINA_T1A_H5_PWM） · 按杰哥 2026 年 11 月最新代码

# 线圈振动<br>一杯咖啡读完

先看全局，再一层层拆开。每一层都配一个例子和对应的代码位置。


1 一句话和一张总图

2 硬件：线圈、引脚、两套一模一样的东西

3 四个定时器，各干各的

4 跟着一次左键按下走一遍（主线）

5 通电多久：时长从哪来

6 怎么通电：正常模式和静音模式

7 PWM 怎么做出“百分之几”

8 力度表怎么一格一格走完

9 振完以后：引脚回低、状态机、霍尔冻结

10 两个线圈不同时震

11 松开时的振动

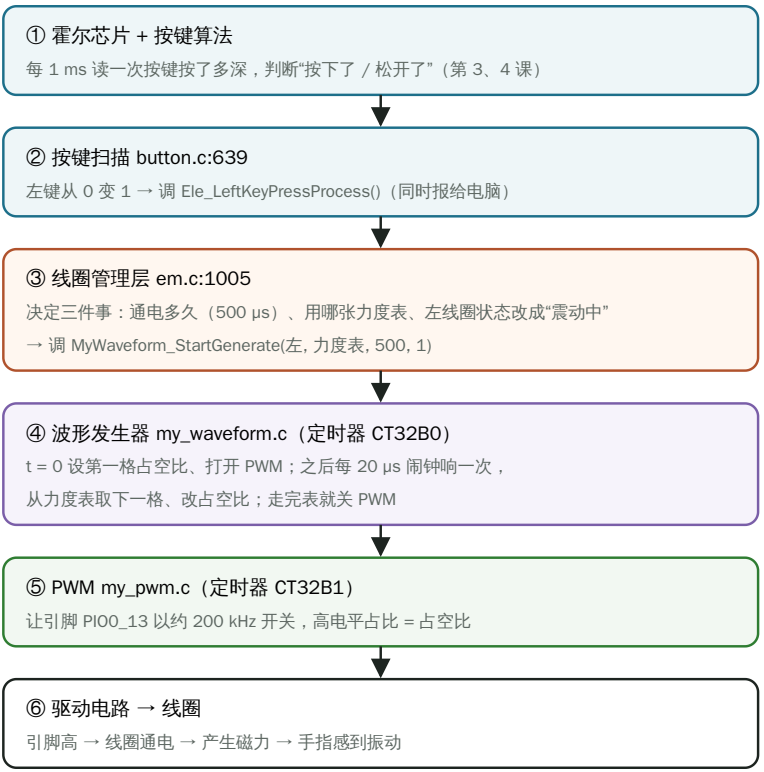
12 容易混的 10 个问题

13 读代码路线和数字速查


行号以仓库代码为准；你编辑器里的行号一般比这里大 1。em.c = src/my_electromagnet.c。

# 1　一句话和一张总图

一句话：按键扫描一确认“左键按下了”，就调 `Ele_LeftKeyPressProcess()`；它算好通电多久、选好力度表，交给波形发生器；波形发生器每 20 μs 改一次 PWM 占空比，PWM 引脚控制线圈通电；时间一到就关掉。线圈通电的那几百微秒里，手指感到“咔”一下。




另外还有一条“旁路”：em.c 的状态机每圈主循环检查一次，
振动期间及之后 9 ms 冻结左键的霍尔读数（第 9 章）。

后面每一章都只是把这张图的某一格放大来看。看糊涂了就翻回这张图，找到自己在哪一格。

# 2　硬件：线圈、引脚、两套一模一样的东西

## 线圈是什么

线圈就是一个小电磁铁。通电时产生磁力，把里面的东西拉一下或撞一下；断电后磁力消失。整个过程只有几百微秒到几毫秒，手指感到的是一下短促的“咔”。

单片机引脚的电流很小，推不动线圈，所以中间有驱动电路：引脚高 → 驱动管导通 → 线圈通电；引脚低 → 线圈断电。线圈的电源来自升压芯片 ETA1628，开机时由 `HP_EN` 引脚打开（em.c:464-466）。

> 记住：软件能控制的只有一件事——这根引脚什么时候高、什么时候低。振动的“长短”和“轻重”，全都是靠控制高低电平做出来的。

## 左右两套，一一对应

| | 左键线圈 | 右键线圈 |
|---|---|---|
| 引脚 | PIO0_13 | PIO0_14 |
| 引脚功能（my_pwm.c:76-81） | CT32B1_MAT0 | CT32B1_MAT1 |
| PWM 比较寄存器 | CT32B1 的 MR0 | CT32B1 的 MR1 |
| 波形发生器通道 | CHANNEL_0 → `wfCh0` | CHANNEL_1 → `wfCh1` |
| 20 μs 闹钟 | CT32B0 的 MR0 | CT32B0 的 MR1 |
| 线圈句柄（状态、时长） | `leftEle` | `rightEle` |
| 按下 / 松开函数 | Ele_LeftKeyPress/ReleaseProcess | Ele_RightKeyPress/ReleaseProcess |

两边代码几乎逐行相同，只是换了名字。下面全部只讲左键，右键照着换名字就行。

举个例子

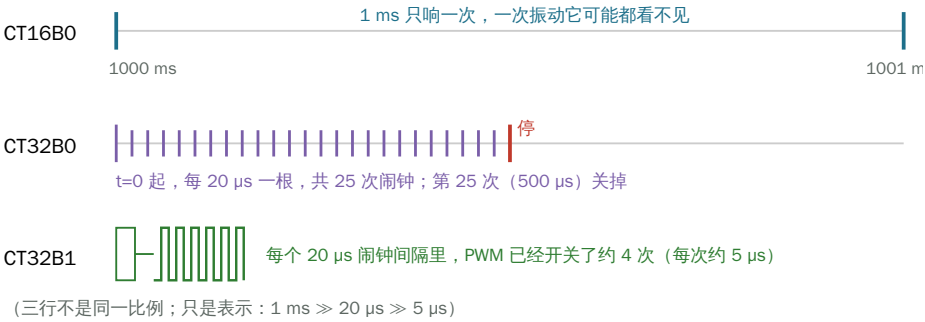
看到 `MR0` 要先问“哪个定时器的 MR0”：CT32B1 的 MR0 管左线圈的占空比；CT32B0 的 MR0 是左线圈的 20 µs 闹钟。两个完全不是一回事。

## 3　四个定时器，各干各的

线圈振动最容易乱的地方，是好几个定时器搅在一起。先把它们分开：

| 定时器 | 比喻 | 多快 | 什么时候在跑 | 在线圈里干什么 |
|---|---|---|---|---|
| CT16B0<br>Timer.c:65-73 | 墙上的挂钟 | 1 ms 一次中断 | 开机后一直跑 | 给 `mySysTickMs` 加 1。状态机用它算“震了几毫秒、消抖几毫秒” |
| CT32B0<br>my_waveform.c:68 | 振动时才上的闹钟 | TC 每 1 µs 加 1；闹钟每 20 µs 响一次 | TC 一直数；闹钟只在振动时打开 | 每 20 µs 从力度表取下一格、改占空比 |
| CT32B1<br>my_pwm.c:72 | 飞快的开关 | 48 MHz 计数，241 下一个周期 ≈ 5 µs（≈ 199 kHz） | 一直数；引脚输出只在振动时打开（PWMC） | 按 MR0 让引脚高低交替，做出占空比 |
| CT16B1 | 秒表 | µs 级延时 | 需要时 | 线圈不用它 |

三个时间尺度套在一起（一次 500 µs 的振动）




（三行不是同一比例；只是表示：1 ms ≫ 20 µs ≫ 5 µs）

举个例子

CT32B0 的 TC 从开机起一直往上数，从不清零。“每 20 μs 响一次”不是定时器自己周期复位，而是每次中断里手动把 MR0 往后挪 20（my_waveform.c:242 加 20，:98 写回 MR0）。所以闹钟只在振动时响；振完把 MR0 中断关掉（:311），它就不响了。

CT16B0 不一样：它开了“匹配后复位”，1 ms 一次，永远在响。

## 4 跟着一次左键按下走一遍（主线）

设定：GUI 全默认（正常模式、导通 500 μs、动态导通关）。时刻 `mySysTickMs = 1000` 时，按键扫描确认左键按下。

1. 按键扫描确认按下，调线圈函数

```
if(0 == i)  Ele_LeftKeyPressProcess();      // button.c:639
```

同一时刻 button_changed = TRUE，USB 报告照常发给电脑。振动和报告互不等待。

2. 检查两件事：要不要震，能不能震

```
if(leftEle.pressVibrateDurationUs)                // em.c:1010  500≠0 → 要震
  if(ELE_PRESS_VIBRATING == rightEle.status || ...) // :1013 右线圈在震吗？
      { 记下来，稍后补；return; }                    // :1015-1018（第 10 章）
```

例子里右边空闲，继续往下走。

3. 算通电多久

```
wfPeriodUs = Ele_CalculateDuration(左, nowHallSegment[0], 500); // :1022 → 500
```

动态导通关着，原样返回 500（第 5 章）。

4 改状态、记时刻

```
leftEle.status = ELE_PRESS_VIBRATING;     // :1023 左线圈"按下震动中"
leftEle.vibrateBeginTick = 1000;          // :1024 开始于 1000 ms
```

从这一刻起，左键的霍尔读数被冻结（第 9 章）。

5 选力度表

```
if(Config_GetCoilDriveMode())   pLeftWf = &wfCustom (→ gPwmWf, 100 格); //
:1025-1029 静音
else                            pLeftWf = &wfHigh   (→ {100}, 1 格);    //
:1033 正常（本例）
```

6 交给波形发生器

```
MyWaveform_StartGenerate(CHANNEL_0, pLeftWf, 500, 1);  // :1035
```

它算好：更新 500÷20 = 25 次（my_waveform.c:179）；然后 Start：t=0 设占空比 100%、打开 PWM、定闹钟 t=20 µs（:283-289）。线圈从这一刻开始通电。

7 每 20 µs 闹钟响一次（中断 my_waveform.c:87 → :233）

t = 20, 40 ⋯ 480 µs：每次取表里的下一格。正常模式表只有 1 格，取的一直是 100%。

t = 500 µs：第 25 次，走出表 → `MyWaveform_Stop`（:249）关 PWM，线圈断电，引脚回低。

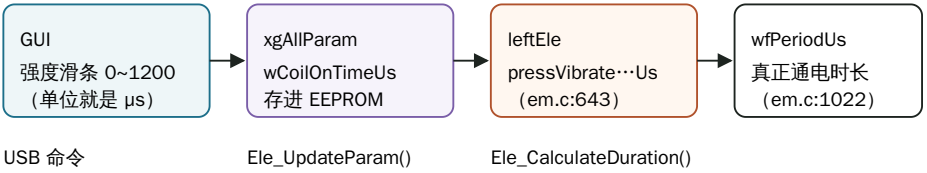
8 主循环里的状态机收尾（em.c:1367）

1002 ms：震动状态结束 → 改成"消抖"；1011 ms：消抖结束 → 改回"空闲"。左键霍尔读数恢复（第 9 章）。

注意第 6~7 步和第 8 步是两套独立的计时：真正断电是波形发生器在 500 µs 时做的（硬件中断，精确到 µs）；状态机用毫秒挂钟，只是"记账"，晚一点才把状态改掉，它不负责断电。

# 5　通电多久：时长从哪来

## 从 GUI 到 leftEle




默认 500（DEFAULT_CFG_ONTIME_US）；上限 5000（MAX_VIBRATE_DURATION_US）；GUI 关掉该项 → 0 → 不震

按下和松开各有一个时长：`pressVibrateDurationUs`、`releaseVibrateDurationUs`。左右键各一套，所以一共 4 个数。

## Ele_CalculateDuration：要不要按深度缩短

```
if(xCoilDebugCfg[index].bDynOnTimeEn)                          // em.c:1348 动态导通开关
    duty = (settingDuration * durationScale[segment]) >> 10;  // :1352 设定 × 系数
else
    duty = settingDuration;                                    // :1361 原样
```

`durationScale[s] = 102400 ÷ (2s + 100)` 是提前算好的表（:210-221），`>> 10` 是 ÷1024。合起来就是：时长 = 设定 × 100 ÷ (2 × 段号 + 100)。段号是按下那一刻按键被按到第几段（0~100）。

举个例子

设定 500 μs，动态导通打开：

段号 0 → 500 × 1024 ÷ 1024 = 500；段号 50 → 500 × 512 ÷ 1024 = 250；段号 100 → 500 × 341 ÷ 1024 = 166。

按得越深，震得越短。但默认是关的，而且新版 GUI 隐藏了这个开关，所以平时时长 = GUI 设的值。

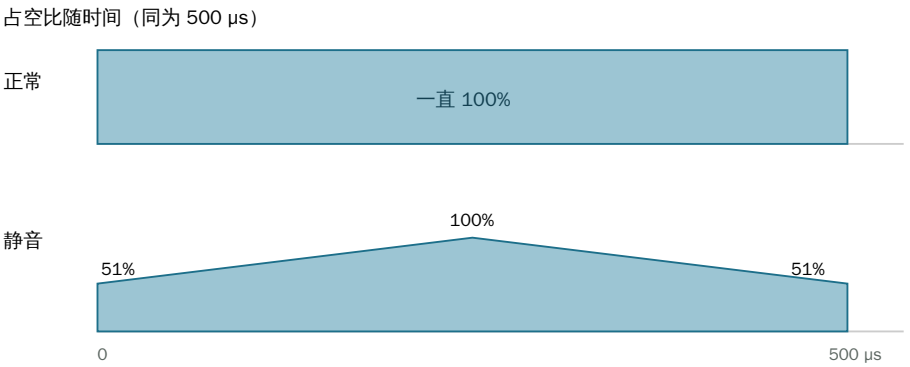
### 静音挡位

新版 GUI 选“静音模式”后，挡位 0~3 直接把左右键按下 / 松开 4 个时长一起设成 0 / 1500 / 2000 / 2500 μs，同时驱动方式设成 pwm。挡位 0 = 时长 0 = 不震。

## 6　怎么通电：正常模式和静音模式

时长决定“震多久”，驱动方式决定“这段时间里力度怎么变”。驱动方式是全局一个：`xgAllParam.xDriveMode.bDriveMode`，左右键、按下松开都用它。

| | 正常模式（level） | 静音模式（pwm） |
|---|---|---|
| bDriveMode | 0（默认） | 非 0 |
| 力度表 | `wfHigh` → `wfHighTbl[] = {100}`，1 格 | `wfCustom` → `gPwmWf[100]`，100 格 |
| 表的内容 | 从头到尾 100% | 51% → 100% → 51% |
| 电流 | 一下冲上去，一下断掉 | 慢慢升，慢慢降 |
| 手感 / 声音 | 脆、响 | 软、轻 |




关键：两种模式走的是同一条路——都通过 PWM、都通过波形发生器。正常模式不是“不用 PWM”，而是用一张只有一格 100% 的表，PWM 占空比 100% 就等于引脚一直高。

### 那张 100 格的表

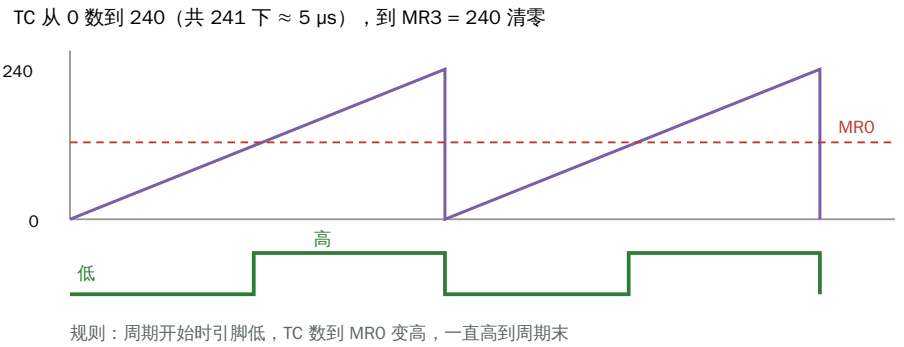
```
for(i = 0; i < 50; i++){                        // my_protocol_mouse.c:181-185
  gPwmWf[i]      = i + 51;     // [0]=51 … [49]=100
  gPwmWf[99 - i] = i + 51;     // [99]=51 … [50]=100
}
```

它在全局参数 `xgAllParam` 里，存 EEPROM。GUI 原本可以通过“PWM配置”面板改它，新版 GUI 隐藏了这个面板，所以一直是默认值。`Config_GetPwmTable()` 返回的就是它的首地址，`wfCustom.pWfData` 指向它。

## 7　PWM 怎么做出“百分之几”

引脚只有高和低两种状态。所谓“51% 的力”，是让引脚在每个约 5 μs 的小周期里，有 51% 的时间是高的。线圈电流跟不上这么快的开关，只能体现出平均值，所以等效成一半左右的力。

### CT32B1 一个周期里发生什么




所以 MR0 越小，高电平越长。高电平占比 = (241 − MR0) ÷ 241。代码用一张查表 `Freq200kHzDuty[101]`（my_pwm.c:19）把“百分比 p”换成 MR0：

| 占空比 p | MR0 = Freq200kHzDuty[p] | 结果 |
| --- | --- | --- |
| 0% | 241（比 240 还大，永远数不到） | 一直低，不通电 |
| 51% | 118 | 低 118 下、高 123 下 → 123/241 ≈ 51% |
| 53% | 113 | 高 128 下 ≈ 53% |
| 100% | 0 | 一直高（正常模式就是这个） |

```
static inline void MyPWM_SetDuty(uint8_t channel, uint8_t percent)  // my_pwm.h:39
{ percent = percent > 100 ? 100 : percent;
  MyPWM_CT32B1_SetCompare(channel, Freq200kHzDuty[percent]); }    // 写 MR0 或 MR1
```

### PWM 的开和关

| 动作 | 代码 | 效果 |
| --- | --- | --- |
| 开机 | MR3=240，PWMC=0，EMC0=01，TC 开始数（my_pwm.c:72-96、:35-37） | 计数器一直在转，但引脚不受 PWM 控制，保持低 |
| 开始震 | `PWMC \|= 1`（MyPWM_CT32B1_Start） | 引脚交给 PWM，按 MR0 开关 |
| 停止震 | `PWMC &= ~1`（MyPWM_CT32B1_Stop） | 引脚交回 EMR 控制；EMC0=01 表示“匹配时清 0”，下一次匹配就把引脚拉低并保持 |

举个例子

“开关 PWM”不是开关计数器——CT32B1 的计数器开机后一直在数。开关的只是 PWMC 里的一位：这根引脚听不听 PWM 的。

## 8　力度表怎么一格一格走完

表有 100 格（正常模式 1 格），时长可能是 500，也可能是 2500。波形发生器要把表铺满整段时长。办法是三句话：

```
updateTimes = periodUs / 20;                    // my_waveform.c:179 闹钟响几次
inc = (wfDataLen << 15) / updateTimes;          // :195 每次走几格（×32768）
acc = inc >> 1;                                 // :196 起点：半步
```

每次闹钟响（:237-241）：`acc += inc`；`idx = acc >> 15`（÷32768 去掉小数）；用第 idx 格设占空比。idx 到 100 就停。

> 举个例子
>
> 500 μs：闹钟 25 次，每次走 100÷25 = 4 格，起点 2 格。
> t = 0, 20, 40 … 480 μs 用的格：2, 6, 10 … 98；t = 500 走到 102，出界，停。
>
> 1500 μs：闹钟 75 次，每次走 1.333 格，起点 0.667 格。
> 用的格：0, 1, 3, 4, 5, 7, 8 … 99（第 2 格这种偶尔被跳过）；t = 1500 出界，停。
>
> 25 和 75 是两个不同时长的例子，不是同一次振动。时长越长，闹钟响得越多，每次走得越少，走过的都是同一条 51→100→51 的曲线。

## 为什么要乘 32768

1.333 格存不进整数。把一格当成 32768 份：每次走 43690 份，acc 记“走到第几份”，`>> 15` 就是“在第几格”。这是单片机常用的“定点小数”，避免用浮点和除法（M0 内核没有硬件除法）。

## 正常模式（1 格）也走这套

格数 = 1，inc = 32768 ÷ 25 = 1310 份。acc 要走 25 步才超过 32768，所以前 25 次一直是第 0 格（100%），第 25 次（t = 500）出界、停。同一套代码，表只有 1 格就等于“通电 500 μs”。

## 实际通电时间

正好是 updateTimes × 20 μs。时长不是 20 的倍数时，余数被 :179 的整数除法丢掉：设 1010 → 实际 1000。

# 9　振完以后：引脚回低、状态机、霍尔冻结

## 引脚回低

走出表时 `MyWaveform_Stop`（my_waveform.c:305）：关 PWM（PWMC 清 0）、关 CT32B0 的 MR0 中断。引脚交回 EMR，在下一次匹配时被拉低并保持。线圈断电。

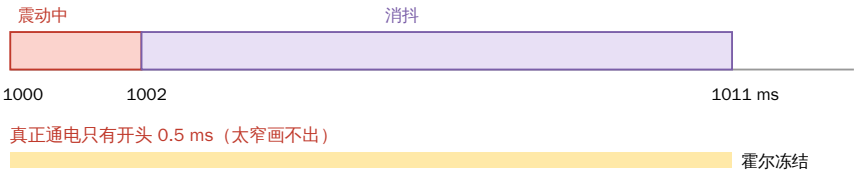
## 状态机：线圈的四个状态

| 值 | 名字 | 意思 |
|---|---|---|
| 0 | ELE_NO_VIBRATING | 空闲 |
| 1 | ELE_PRESS_VIBRATING | 按下震动中 |
| 2 | ELE_RELEASE_VIBRATING | 松开震动中 |
| 3 | ELE_RELEASE_DEBOUNCE | 刚震完，等余振平息（消抖） |

```
case ELE_PRESS_VIBRATING:                                          // em.c:1375
  if((now - vibrateBeginTick) * 1000 > pressVibrateDurationUs + 999) // :1377
      { status = ELE_RELEASE_DEBOUNCE; debounceBeginTick = now; }
case ELE_RELEASE_DEBOUNCE:
  if((now - debounceBeginTick) > 8)  status = ELE_NO_VIBRATING;     // :1394
VIBRATE_DEBOUNCE_TIME_MS
```

谁调：主循环（device.c:525）每圈 → `Ele_PollingTask`（em.c:983）→ `Ele_StatusTask`，左右各一次。

> 举个例子
>
> 时长 500，按下于 1000 ms：
> 震动状态：要 (now − 1000) × 1000 > 1499 → now − 1000 ≥ 2 → 1002 ms 进消抖。
> 消抖：要 now − 1002 > 8 → ≥ 9 → 1011 ms 回空闲。
> 时长 1500：(n × 1000 > 2499 → n ≥ 3) → 1003 ms 进消抖，1012 ms 回空闲。




### 为什么要冻结霍尔

```
if(Ele_IsVibrating(ELE_POS_LEFT))       // my_misc.c:490 状态不是“空闲”
  if(k == 1) { gbKeyState[k] = previousKeySta[k]; continue; }   // 左键保持上一次的结果
```

线圈通电会产生磁场，霍尔芯片测的正是磁场，读数会被干扰；震完还有余振。如果这时照常判断，可能误判成“松开了”又“按下了”，导致连震。所以从开始震到消抖结束（例子里 1000~1011 ms），左键的按键状态保持不变。

两个查询函数别混：`Ele_IsVibrating`（em.c:917）= 不是空闲（含消抖），用于冻结霍尔；`Ele_IsTureVibrating`（:949）= 正在通电（按下或松开震动中，不含消抖）。

## 10　两个线圈不同时震

T1A 编译了 `SINGLE_COIL_AT_A_TIME = 1`：同一时刻只让一个线圈通电（两个一起通电电流太大）。

```
if(ELE_PRESS_VIBRATING == rightEle.status || ELE_RELEASE_VIBRATING == rightEle.status)
{ gxCacheVibrate.flag = 1;  .pos = ELE_POS_LEFT;  .tick = now;  return; }   // em.c:1013-1018
```

左键要震时，如果右线圈正在通电（消抖不算），就先不震，把“左边欠一次”记在 `gxCacheVibrate` 里。

```
if(gxCacheVibrate.flag && (now - gxCacheVibrate.tick) > 5)    // em.c:1473-1475
(5000/1000)
{ flag = 0;  if(pos == LEFT) Ele_LeftKeyReleaseProcess(); ... }  // :1480
```

主循环里的 `Ele_OnlyOne`（em.c:1471）过了 5 ms 以上（即 ≥ 6 ms）补震。

> 举个例子
>
> 右键 1000 ms 按下开始震；1000 ms 同时左键也按下 → 左边被记下 tick = 1000。1006 ms 补一次左边。
>
> 注意：补的时候调的是松开函数，用的是松开的时长和状态，即使欠的是“按下”。这一点已列在待确认问题第 14 条。

## 11　松开时的振动

松开和按下几乎一模一样，只有三处不同：

| | 按下 em.c:1005 | 松开 em.c:1066 |
|---|---|---|
| 谁调 | button.c:639（0 → 1） | button.c:655（1 → 0） |
| 用哪个时长 | pressVibrateDurationUs | releaseVibrateDurationUs |
| 状态改成 | ELE_PRESS_VIBRATING | ELE_RELEASE_VIBRATING |
| 力度表、波形发生器、PWM、状态机、霍尔冻结 | 完全相同 | |

> 举个例子
>
> 一次完整的点击 = 两次振动：按下“咔”一下，松开再“咔”一下。GUI 里“按下振感配置”“释放振感配置”就是分别设这两个时长。

按住不放时不会连震：按下震一次后，按键状态一直是 1，button.c 只在 0→1、1→0 的那一下调线圈函数。

## 12　容易混的 10 个问题

> Q1 “25 次”和“75 次”到底是哪个？
> 都对，是两个不同时长：500 μs → 25 次；1500 μs → 75 次。次数 = 时长 ÷ 20。

> Q2 线圈是靠什么断电的？状态机吗？
> 不是。断电是波形发生器走出表时 `MyWaveform_Stop` 做的，μs 级精确。状态机只按毫秒“记账”，用来冻结霍尔和防止马上又震。

> Q3 正常模式用不用 PWM？
> 用。只是表是 {100}，占空比 100% → MR0 = 0 → 引脚一直高。

> Q4 力度表管长短吗？
> 不管。表只管形状；长短只看 wfPeriodUs。同一张表被拉长或压短。

> Q5 MR0 是谁的？
> CT32B1 的 MR0 = 左线圈占空比；CT32B0 的 MR0 = 左线圈 20 μs 闹钟。右线圈对应两个 MR1。

> Q6 PWM 为什么是 241 下？
> MR3 = 240，TC 数 0~240 共 241 下，48 MHz 下约 5.02 μs，≈ 199 kHz。

> Q7 动态导通开着时，时长是软件自己定的吗？
> 不是。仍然以 GUI 设的值为基础，只是乘一个 ≤ 1 的系数（按得越深越小）。默认关，新 GUI 也打不开。

> Q8 振完引脚为什么能保证是低？
> 停 PWM 只清 PWMC，引脚交回 EMR；开机时把 EMC0 设成 01（匹配时清 0），所以下一次匹配就被拉低并保持。

> Q9 振动会耽误按键报给电脑吗？
> 不会。button.c 在同一处既设 button_changed（报告）又调线圈函数，线圈函数只是设好寄存器就返回，后面全靠中断。

> Q10 CT16B0、CT32B0、CT32B1 一句话怎么分？
> CT16B0 = 1 ms 挂钟（状态机记账）；CT32B0 = 振动时的 20 µs 闹钟（换格）；CT32B1 = 5 µs 一周期的开关（做占空比）。

# 13　读代码路线和数字速查

## 按执行顺序读，一共 9 站

| # | 位置 | 看什么 |
|---|---|---|
| 1 | em.c:439 Ele_Init → my_waveform.c:135 → my_pwm.c:33、:72；my_waveform.c:68 | 开机：CT32B1 配成 PWM 但不输出；CT32B0 开始 1 µs 计数 |
| 2 | button.c:630-660 | 按下 / 松开时调哪个函数 |
| 3 | em.c:1005-1065 | 按下函数：要不要震、时长、状态、选表、启动 |
| 4 | em.c:1327-1365 | Ele_CalculateDuration |
| 5 | my_waveform.c:155-202 | StartGenerate：次数、inc、acc |
| 6 | my_waveform.c:278-291 | Start：t=0 设占空比、开 PWM、定闹钟 |
| 7 | my_waveform.c:87-99、:233-259 | 中断：每 20 µs 走一步；走出就 Stop |
| 8 | em.c:983、:1367-1403、:1471 | 主循环：状态机、补震 |
| 9 | my_misc.c:488-510 | 霍尔冻结 |

## 数字速查

| 数 | 含义 | 出处 |
|---|---|---|
| 500 μs | 默认导通时间 | my_electromagnet.h:26 |
| 5000 μs | 导通时间上限；补震等待 5 ms | em.c:27；my_waveform.c:22 |
| 20 μs | 波形更新间隔（闹钟） | my_waveform.c:21 |
| 32768（<<15） | 1 格 = 32768 份 | my_waveform.c:25 |
| 100 | 静音力度表格数 | em.c:1028 |
| 51→100→51 | 默认力度表 | my_protocol_mouse.c:181 |
| 240 / 241 | PWM 周期 MR3 / 计数个数 | my_pwm.c:35 |
| 8 ms（实际 9） | 震后消抖 | em.c:29 |
| 0 / 1500 / 2000 / 2500 | 静音挡位 0~3 的时长 | 新版 GUI |
| 0~1200 | GUI 强度滑条（μs） | 新版 GUI |

喝完咖啡再回想一遍：按键确认 → 线圈函数定时长和表 → 波形发生器每 20 μs 换一格 → PWM 把百分比变成引脚高低 → 走出表就断电 → 状态机记账、冻结霍尔约 11 ms。